



รายงานปฏิบัติการ PS07: Concurrency and Thread

เสนอ

อาจารย์ ราชวิรัช สโรชวิกสิต

จัดทำโดย

นายภูมิพัฒน์ อภิวัตนะพงศ์	67070501035
นายวิรัช ทองอุทัยศรี	67070501041
นายเจษฎา เกียรติกมลวงค์	67070501080

รายงานนี้เป็นส่วนหนึ่งของวิชา CPE 333 Operating Systems
ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์
ภาคการศึกษาที่ 1 ปีการศึกษา 2569
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

Contents

สภาพแวดล้อมที่ใช้ในการทดลอง	1
1 โปรแกรมที่ใช้ทดลอง	1
2 Task 1: รันด้วย thread เดียว	1
3 Task 2: หลาย thread โดยไม่บังคับ context switch	2
4 Task 3: บังคับ context switch	3
5 อภิปรายผลและสาเหตุของ race condition	4
สรุปผลการทดลอง	5

สภาพแวดล้อมที่ใช้ในการทดลอง

ทดลองบน Ubuntu 24.04.1 LTS ที่ทำงานบน WSL2 (kernel 6.6.87.2-microsoft-standard-WSL2) CPU คือ Intel Core Ultra 9 185H ซึ่ง Linux มองเห็นเป็น 22 logical CPU คอมไพเลอร์คือ gcc 13.3.0 และ thread library คือ POSIX threads ของ glibc 2.39 ผลการรันทุกครั้งในรายงานเป็นภาพหน้าจอจริงจาก terminal และตัวเลขทุกตัวอ่านจากภาพเหล่านั้น

1 โปรแกรมที่ใช้ทดลอง

ตัวอย่างที่เลือกคือบัญชีเงินฝากร่วมหนึ่งบัญชี ยอดเงินอยู่ในตัวแปร global balance (เริ่มต้น 1,000 บาท) ซึ่งตู้ ATM หลายตู้ใช้ร่วมกัน แต่ละตู้คือ thread หนึ่งตัว thread หนึ่งตู้หนึ่งฝากเงินและอีกตู้หนึ่งถอนเงิน ครั้งละ 1 บาท จำนวนครั้งเท่ากัน ยอดสุดท้ายที่ถูกต้องจึงต้องเป็น 1,000 บาทเสมอ ต่างจากตัวอย่างในคาบเรียนที่เพิ่มค่าตัวนับทางเดียว ตัวอย่างนี้มีทั้งเพิ่มและลด ผลที่ผิดจึงเป็นไปได้ทั้งบวก (รายการถอนหาย) และลบ (รายการฝากหาย) การฝากและถอนแก้ค่า balance ตรง ๆ โดยไม่มี lock ป้องกัน ซึ่งเป็นจุดที่ทำให้เกิด race condition

```
1 void deposit_plain(void)
2 {
3     balance = balance + AMOUNT;
4 }
5
6 void withdraw_plain(void)
7 {
8     balance = balance - AMOUNT;
9 }
```

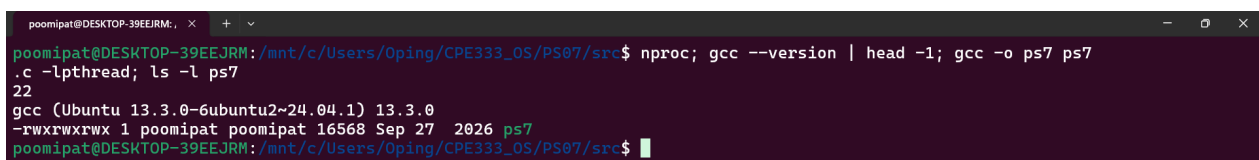
Code 1: การฝากและถอนเงินที่ไม่มีการป้องกัน

วิธีใช้คือ ./ps7 [threads] [tx_per_thread] [yield] ถ้าระบุ 1 thread ตัวนั้นจะฝาก tx ครั้งแล้วจึงถอน tx ครั้ง ถ้าระบุเป็นเลขคู่ thread ครั้งแรกฝาก ครั้งหลังถอน ตัวละ tx ครั้ง main() สร้าง thread ทั้งหมดด้วย pthread_create() ก่อนแล้วจึงรอด้วย pthread_join() เพื่อให้ทุกตัวทำงานซ้อนกัน แต่ละ thread นับรายการที่ทำเสร็จไว้ในตัวแปร local ของตัวเอง ซึ่งไม่ได้ใช้ร่วมกันจึงถูกต้องเสมอ ใช้ยืนยันว่ารายการครบทุกรายการ

แต่ละรอบพิมพ์ผลหนึ่งบรรทัด in และ out คือจำนวนรายการฝากและถอนที่ทำจริง expected คือยอดที่ควรเป็น (1,000) actual คือค่าใน balance และ diff คือ actual - expected ท้ายบรรทัดเป็น OK เมื่อยอดถูกต้อง และ RACE เมื่อยอดผิด

คอมไพล์ด้วยคำสั่งตามใบงาน gcc -o ps7 ps7.c -lpthread โดยตั้งใจไม่ใส่ -o เพราะ -o2 อาจรวบรวบทั้งรูปแบบการบวกครั้งเดียว ทำให้ race condition แทบไม่ปรากฏ

2 Task 1: รันด้วย thread เดียว



```
poomipat@DESKTOP-39EEJRM: ~$ gcc --version | head -1; gcc -o ps7 ps7.c -lpthread; ls -l ps7
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
-rwxrwxrwx 1 poomipat poomipat 16568 Sep 27 2026 ps7
poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$
```

Figure 1: จำนวน CPU รุ่นของ gcc และการคอมไพล์ด้วยคำสั่งตามใบงาน

```
poomipat@DESKTOP-39EEJRM: . x + v
poomipat@DESKTOP-39EEJRM:/mnt/c/Users/Oping/CPE333_OS/PS07/src$ for i in $(seq 5); do ./ps7 1 1000000; done
[plain] threads=1 tx=1000000 in=1000000 out=1000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=1 tx=1000000 in=1000000 out=1000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=1 tx=1000000 in=1000000 out=1000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=1 tx=1000000 in=1000000 out=1000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=1 tx=1000000 in=1000000 out=1000000 | expected=1000 actual=1000 diff=+0 OK
poomipat@DESKTOP-39EEJRM:/mnt/c/Users/Oping/CPE333_OS/PS07/src$
```

Figure 2: Task 1: รันด้วย thread เดียว 5 รอบ ได้ยอดเงินถูกต้องทุกรอบ

คอมไพล์ผ่านโดยไม่มี warning และเมื่อรันด้วย thread เดียว ทั้ง 5 รอบได้ actual=1000 diff=+0 OK เหมือนกันทุกรอบ เพราะมี thread เพียงตัวเดียวที่แก้ balance การอ่าน-บวก-เขียนแต่ละครั้งจึงจบก่อนเริ่มครั้งถัดไปเสมอ ผู้เขียนยืนยันว่าตรรกะของโปรแกรมถูกต้อง ถ้าใช้หลาย thread แล้วยอดผิด สาเหตุจึงมาจากการแก้ตัวแปรร่วมกันเท่านั้น

3 Task 2: หลาย thread โดยไม่บังคับ context switch

แก้ไขให้ใช้หลาย thread โดยสั่ง ./ps7 4 1000000 คือฝาก 2 ตัวและถอน 2 ตัว ตัวละ 1,000,000 รายการ ยอดที่ถูกต้องยังเป็น 1,000 บาท รันสองแบบ คือแบบปกติที่ thread กระจายอยู่บน CPU 22 ตัว และแบบที่ใช้ taskset -c 0 บังคับให้ทุก thread อยู่บน CPU ตัวเดียว

```
poomipat@DESKTOP-39EEJRM: . x + v
poomipat@DESKTOP-39EEJRM:/mnt/c/Users/Oping/CPE333_OS/PS07/src$ for i in $(seq 10); do ./ps7 4 1000000; done
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=-32229 diff=-33229 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=-130619 diff=-131619 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=152202 diff=+151202 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=121915 diff=+120915 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=140707 diff=+139707 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=-6078 diff=-7078 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=-113785 diff=-114785 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=-725602 diff=-726602 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=380062 diff=+379062 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=-186190 diff=-187190 RACE
poomipat@DESKTOP-39EEJRM:/mnt/c/Users/Oping/CPE333_OS/PS07/src$
```

Figure 3: Task 2: 4 thread บน CPU 22 ตัว รัน 10 รอบ ผิดทุกรอบและไม่ซ้ำกันเลย

บน CPU 22 ตัว ผิดทั้ง 10 รอบและไม่ซ้ำกันเลย บวก 4 รอบ ลบ 6 รอบ ตั้งแต่ -726,602 ถึง +379,062 ทั้งที่ in=2000000 out=2000000 ครบทุกบรรทัด แปลว่ารายการถูกทำครบ แต่ผลของการเขียนลง balance หายไป สาเหตุคือ thread ได้ CPU คนละตัวและทำงานพร้อมกันจริง จึงชนกันได้ในทุกรอบของลูป

```
poomipat@DESKTOP-39EEJRM: . x + v
poomipat@DESKTOP-39EEJRM:/mnt/c/Users/Oping/CPE333_OS/PS07/src$ for i in $(seq 20); do taskset -c 0 ./ps7 4 1000000; done
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=2001000 diff=+1000000 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1001000 diff=+1000000 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1001000 diff=+1000000 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=-628150 diff=-629150 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1001000 diff=+1000000 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1001000 diff=+1000000 RACE
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=1001000 diff=+1000000 RACE
poomipat@DESKTOP-39EEJRM:/mnt/c/Users/Oping/CPE333_OS/PS07/src$
```

Figure 4: Task 2: 4 thread บน CPU ตัวเดียว รัน 20 รอบ ถูก 13 รอบ ผิด 7 รอบ

บน CPU ตัวเดียว ถูก 13 รอบ ผิด 7 รอบ และค่าที่ผิดเป็นก้อนลงตัว คือ +1,000,000 (5 รอบ), +2,000,000 และ -629,150 กรณีนี้ thread สลับกันได้เฉพาะเมื่อ kernel ขัดจังหวะ รอบส่วนใหญ่จึงถูก เพราะลูปจบก่อนโดนขัดหรือโดนขัดนอกช่วงอ่าน-เขียน แต่ถ้า thread ฝากเงินโดนขัดหลังอ่าน balance แล้วแต่ยังไม่ได้เขียนกลับ ระหว่างรอ

thread ถอนเงินอาจถอนครบทั้ง 1,000,000 ครั้ง แล้ว thread ฝากก็กลับมาเขียนค่าเก่าทับ ได้ +1,000,000 พอดี นี่คือ untimely context switch ที่เกิดขึ้นเองแต่ไม่บ่อย จึงต้องรันหลายครั้งถึงจะเห็น

```
poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$ objdump -d --no-show-raw-insn -M intel ps7 | awk '/<deposit|withdraw_plain>:/{ret}'
000000000001269 <deposit_plain>:
1269:  endbr64
126d:  push  rbp
126e:  mov   rbp, rsp
1271:  mov   rax, QWORD PTR [rip+0x2d98]      # 4010 <balance>
1278:  add   rax, 0x1
127c:  mov   QWORD PTR [rip+0x2d8d], rax      # 4010 <balance>
1283:  nop
1284:  pop   rbp
1285:  ret
000000000001286 <withdraw_plain>:
1286:  endbr64
128a:  push  rbp
128b:  mov   rbp, rsp
128e:  mov   rax, QWORD PTR [rip+0x2d7b]      # 4010 <balance>
1295:  sub   rax, 0x1
1299:  mov   QWORD PTR [rip+0x2d70], rax      # 4010 <balance>
12a0:  nop
12a1:  pop   rbp
12a2:  ret
poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$
```

Figure 5: คำสั่งเครื่องของ deposit_plain() และ withdraw_plain()

ภาพจาก objdump แสดงว่า balance = balance + AMOUNT ไม่ใช่คำสั่งเดียว แต่เป็นสามคำสั่ง คือ load (mov rax, [balance]) ที่ 1271, add ที่ 1278 และ store (mov [balance], rax) ที่ 127c ถ้า thread อื่นเขียน balance ในช่วงระหว่าง load กับ store ค่าที่ store ลงไปจะทับผลของ thread นั้น

4 Task 3: บังคับ context switch

เพิ่มโหมด yield ที่แยกการแก้ค่าเป็นสามขั้นตามตัวอย่างในใบงาน และเรียก sched_yield() คั่นก่อนการเขียน (ไม่ใช่ pthread_yield() เพราะ glibc 2.34 ขึ้นไปเลิกใช้แล้ว) จำนวนรายการลดเหลือ 100,000 ต่อ thread เพราะ sched_yield() เป็น system call ทุกรอบ

```
1 void deposit_yield(void)
2 {
3     long reg = balance;           /* Load */
4     reg = reg + AMOUNT;           /* modify */
5     sched_yield();               /* force a context switch */
6     balance = reg;               /* store */
7 }
8
9 void withdraw_yield(void)
10 {
11     long reg = balance;
12     reg = reg - AMOUNT;
13     sched_yield();
14     balance = reg;
15 }
```

Code 2: การฝากและถอนที่บังคับ context switch ระหว่าง load กับ store

```
poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$ gcc -o ps7 ps7.c -lpthread; for i in $(seq 10); do ./ps7 4 10
0000 yield; done
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=-58119 diff=-59119 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=-22974 diff=-23974 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=31997 diff=+30997 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=10558 diff=+9558 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=11834 diff=+10834 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=25285 diff=+24285 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=-12761 diff=-13761 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=7353 diff=+6353 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=-55357 diff=-56357 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=-59108 diff=-60108 RACE
poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$
```

Figure 6: Task 3: คอมไพล์ใหม่แล้วรันโหมด yield บน CPU 22 ตัว 10 รอบ ผิดทุกรอบ

บน CPU 22 ตัว ผิดทั้ง 10 รอบและไม่ซ้ำกัน บวก 5 รอบ ลบ 5 รอบ ตั้งแต่ -60,108 ถึง +30,997 บนเครื่องนี้ thread แต่ละตัวมักได้ CPU ของตัวเอง ถ้าไม่มี thread อื่นรออยู่ sched_yield() จะไม่สลับจริง แต่ยังทำให้ช่วง

ระหว่าง load กับ store ยาวขึ้น thread บน CPU อื่นจึงเขียนแทรกได้ง่าย

```

poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$ for i in $(seq 10); do taskset -c 0 ./ps7 4 100000 yield; done; for
i in $(seq 10); do taskset -c 0 ./ps7 4 100000 yield; done
[plain] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=1000 diff=+0 OK
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=-98985 diff=-99985 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=101000 diff=+100000 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=101000 diff=+100000 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=-98951 diff=-99951 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=101000 diff=+100000 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=-98998 diff=-99998 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=-98990 diff=-99990 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=101000 diff=+100000 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=101000 diff=+100000 RACE
[yield] threads=4 tx=100000 in=200000 out=200000 | expected=1000 actual=-99000 diff=-100000 RACE
poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$

```

Figure 7: Task 3: CPU ตัวเดียว 100,000 รายการต่อ thread โหมดปกติถูกทิ้ง 10 รอบ โหมด yield ผิดทิ้ง 10 รอบ

ภาพนี้เป็นกราฟแสดงการทดลองแบบควบคุมบน CPU ตัวเดียว ใช้จำนวน thread และจำนวนรายการเท่ากัน ต่างกันแค่การเรียก sched_yield() ผลคือโหมดปกติถูกทิ้ง 10 รอบ แต่โหมด yield ผิดทิ้ง 10 รอบ ยืนยันว่า context switch ที่เกิดระหว่าง load กับ store คือตัวที่ทำให้ค่าถูกเขียนทับ

ค่าที่ผิดในโหมด yield คือ +100,000 พอติ 5 รอบ และใกล้ -100,000 อีก 5 รอบ เพราะ thread ทั้ง 4 ตัวผลัดกันทำทีละรายการ และแต่ละตัวอ่านค่าที่ตัวเองเพิ่งเขียนกลับมาเสมอ thread ฝาสองตัวจึงรวมกันเป็นต้นฉบับตัวเดียวที่เพิ่มรอบละ 1 ส่วน thread ถอนสองตัวลดรอบละ 1 สองฝั่งเขียนทับกันตลอด ยอดสุดท้ายจึงเป็นค่าของฝั่งที่เขียนคนสุดท้าย คือ 101,000 หรือ -99,000

5 อภิปรายผลและสาเหตุของ race condition

race condition ในโปรแกรมนี้เกิดจากเงื่อนไขสามข้อพร้อมกัน ข้อแรก balance เป็นตัวแปร global ที่ทุก thread ใช้ร่วมกัน ข้อสอง balance = balance + AMOUNT ไม่เป็น atomic เพราะเป็นสามคำสั่ง load, add และ store (ภาพ t05) และข้อสาม ไม่มี lock กันไม่ให้หลาย thread เข้า critical section นี้พร้อมกัน ผลคือ **lost update** ดังตัวอย่างในตาราง

Table 1: ลำดับเหตุการณ์ที่ทำให้รายการถอนเงินหายไปหนึ่งรายการ

ลำดับ	thread A (ฝาก)	thread B (ถอน)	balance ในหน่วยความจำ
1	load: rax = 1000		1000
2	add: rax = 1001		1000
3		load: rax = 1000	1000
4		sub: rax = 999	1000
5		store: balance = 999	999
6	store: balance = 1001		1001

ฝากหนึ่งครั้งและถอนหนึ่งครั้ง ยอดควรเป็น 1,000 แต่ได้ 1,001 เพราะ A เขียนค่าที่คำนวณจากข้อมูลเก่าทับผลของ B ถ้าขั้นที่ 5 กับ 6 สลับกัน ยอดจะเป็น 999 แทน ผลจึงขึ้นกับว่า thread ไต store คนสุดท้าย ขั้นที่ 3-5 เกิดได้ทั้งจาก B ทำงานพร้อมกันบน CPU อีกตัว (t03) และจาก A ถูกสลับออกหลังขั้นที่ 2 (t04 และ t07)

ลำดับการสลับถูกกำหนดโดย scheduler และขึ้นกับสิ่งที่โปรแกรมควบคุมไม่ได้ เช่น จังหวะของ timer interrupt, CPU ที่แต่ละ thread ได้ และงานอื่นในเครื่อง ผลจึงเปลี่ยนไปทุกครั้งที่รัน (non-deterministic) ส่วน sched_yield()

ไม่ได้สร้าง bug ขึ้นมาใหม่ เพียงบังคับให้ช่วงระหว่าง load กับ store ถูกแทรกแน่นอนทุกครั้ง แทนที่จะรอให้เกิดเองโดยบังเอิญ

Table 2: สรุปผลการรันทุกชุด (ตัวเลขทั้งหมดอ่านจากภาพหน้าจอ)

ภาพ	โหมด	thread/CPU	tx/thread	รอบ	รอบที่ผิด	diff ที่พบ
t02	plain	1 / 22	1,000,000	5	0	+0 ทุกรอบ
t03	plain	4 / 22	1,000,000	10	10	-726,602 ถึง +379,062 ไม่ซ้ำกัน
t04	plain	4 / 1	1,000,000	20	7	+1,000,000 (5 รอบ), +2,000,000 และ -629,150
t06	yield	4 / 22	100,000	10	10	-60,108 ถึง +30,997 ไม่ซ้ำกัน
t07	plain	4 / 1	100,000	10	0	+0 ทุกรอบ
t07	yield	4 / 1	100,000	10	10	+100,000 (5 รอบ) และ ใกล้เคียง -100,000 (5 รอบ)

สรุปผลการทดลอง

โปรแกรมให้ผลถูกต้องเสมอเมื่อใช้ thread เดียว แต่เมื่อใช้หลาย thread ยอดเงินผิดและเปลี่ยนไปทุกครั้งที่รัน บน CPU หลายตัว race condition เกิดแทบทุกครั้งเพราะ thread ทำงานพร้อมกันจริง ส่วนบน CPU ตัวเดียวเกิดเฉพาะเมื่อ context switch ต่กระหว่าง load กับ store ซึ่งไม่บ่อย และเมื่อบังคับ context switch ด้วย sched_yield() ก็ผิดทุกครั้ง สาเหตุคือการอ่าน-แก้-เขียนตัวแปรที่ใช้ร่วมกันโดยไม่เป็น atomic และไม่มี mutual exclusion ข้อสังเกตสำคัญคือโปรแกรมที่มี race condition อาจรันผ่านได้หลายครั้ง (13 จาก 20 รอบใน t04) การรันแล้วได้ผลถูกต้องไม่ได้พิสูจน์ว่าโปรแกรมถูกต้อง ต้องป้องกัน critical section ด้วยกลไก synchronization เช่น mutex หรือคำสั่ง atomic